

Trabajo Práctico 2-1: Expresiones Lambda

Punto 1: Invocaciones en AprendiendoLambdas

Desde la clase `Main.java`, se realizan llamadas al método sobrecargado `unMetodo` de la clase `AprendiendoLambdas.java`.

Inicialización

Main.java

```
AprendiendoLambdas a = new AprendiendoLambdas();
```

Se crea la instancia de `AprendiendoLambdas` en la **Heap**. La variable `a` es la referencia a este objeto.

1. Primera Invocación: Interfaz B

Main.java

```
a.unMetodo((b) -> {System.out.println("abcd" + b)});
```

- **Método invocado:** `public interface B { void metodo(String b); }`
- Justificación: La lambda recibe un parámetro `(b)` y no retorna ningún valor.

AprendiendoLambdas.java

```
public void unMetodo(B b) { b.metodo("unString"); }
```

Ejecuta `b.metodo("unString")`. El valor literal `"unString"` se busca en el **String Constant Pool** (dentro de la **Heap**); si no existe, se crea. La lambda recibe esta referencia y concatena.

Salida esperada

```
abcdunString
```

2. Segunda Invocación: Interfaz A

Main.java

```
a.unMetodo(() -> System.out.println("abcd"));
```

- **Método invocado:** `public interface A { void metodo(); }`
- Justificación: La lambda no recibe parámetros `()`, coincide con la firma.

AprendiendoLambdas.java

```
public void unMetodo(A a) { a.metodo(); }
```

Ejecuta el cuerpo de la lambda directamente (el `print`).

Salida esperada

```
abcd
```

3. Tercera Invocación: Interfaz B (Inferencia de Tipos)

Main.java

```
a.unMetodo((variable) -> { System.out.println("abcd"); });
```

- **Método invocado:** `public interface B { void metodo(String b); }`

Justificación: El cuerpo no utiliza el parámetro `variable`; la lambda define un único parámetro de entrada y no tiene valor de retorno. Internamente el compilador realiza un **target typing** e infiere que corresponde a la interfaz `B`.

AprendiendoLambdas.java

```
public void unMetodo(B b) { b.metodo("unString"); }
```

Al igual que en el primer caso, el parámetro `variable` recibe la referencia del valor literal `"unString"` desde el String Pool, aunque no se imprima.

Salida esperada

```
abcd
```

4. Cuarta Invocación: Interfaz C

Main.java

```
a.unMetodo((variable) -> {
    System.out.println("abcd");
    return true;
});
```

- **Método invocado:** `public interface C { boolean metodo(String c); }`

Justificación: La lambda recibe un parámetro e incluye un `return true` explícito. El compilador aplica **target typing**: al detectar que la lambda retorna un `boolean` e infiere que corresponde a la interfaz `C`. La lambda imprime `"abcd"` e ignora el contenido de `variable`.

AprendiendoLambdas.java

```
public void unMetodo(C c) { System.out.println(c.metodo("otroString") ?
"true" : "false"); }
```

El valor de retorno (`true`) es evaluado por un operador ternario que imprime `"true"`. Si se cambiara el retorno de la lambda a `false`, la salida cambiaría consistentemente.

Salida esperada

```
abcd
true
```

5. Quinta Invocación: Interfaz D (Tipado y Genéricos)

Main.java

```
a.unMetodo((Long variable) -> {
    System.out.println("abcd");
    return 10L;
});
```

- **Método invocado:** `public interface D<T, R> { R metodo(T d); }`

Justificación: La lambda especifica que su parámetro es de tipo `Long` y retorna un valor `10L` (también `Long`). Encaja con la especialización de la interfaz genérica `D<T, R>` definida en `AprendiendoLambdas` como `D<Long, Long>`.

AprendiendoLambdas.java

```
public void unMetodo(D<Long, Long> d) { d.metodo(10L); }
```

Nota: el valor de retorno de `d.metodo(10L)` no es utilizado ni impreso dentro de `unMetodo`. La única salida proviene del cuerpo de la lambda.

Salida esperada

```
abcd
```

Conclusión

A lo largo de este trabajo práctico, pude analizar las cinco invocaciones al método sobrecargado `unMetodo`, exploré los distintos mecanismos de implementación con las expresiones lambda en Java, entre ellos:

- La *resolución de sobrecarga*: el compilador determina qué versión de `unMetodo` invocar según la firma de la lambda (número de parámetros y tipo de retorno).
- El *target typing*: cuando el tipo del parámetro no se declara explícitamente, el compilador lo infiere a partir del contexto de la interfaz funcional esperada.
- Las *interfaces funcionales genéricas*: la especialización de `D<T, R>` con tipos concretos (`D<Long, Long>`) permite que el compilador valide correctamente el tipo del parámetro y del valor de retorno de la lambda.

El compilador de Java es quien resuelve la correspondencia entre lambdas e interfaces funcionales, simplificando el código sin perder seguridad de tipos.